



ÍNDICES E DESEMPENHO EM CONSULTAS

BANCO DE DADOS NÃO RELACIONAIS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



CONTEÚDO DA AULA

- O que são Índices
- Índices Simples e Compostos
- Índices em
 - Arrays, Campos Textuais, Geoespaciais
- Espaço Ocupado por Índices
- Verificar uso de um Índice
- Remoção de Índices
- Avaliação de Desempenho de Índices



O QUE SÃO ÍNDICES?

- Os índices são estruturas de dados especiais que armazenam uma parte dos dados de uma coleção de forma ordenada, facilitando buscas rápidas.
- Sem Índice vs. Com Índice
 - Sem índice: O MongoDB precisa percorrer todos os documentos para encontrar os resultados (**full collection scan**).
 - Com índice: O MongoDB acessa diretamente os documentos relevantes, **reduzindo o tempo de busca**.
- Desvantagem dos índices: **Eles ocupam espaço extra em disco e afetam a performance de escrita**, pois sempre que um documento é inserido/atualizado, os índices precisam ser ajustados.

ÍNDICES SIMPLES

Usando apenas um campo

CRIANDO ÍNDICES SIMPLES

- Por padrão, o MongoDB já cria um índice no campo `_id`, mas podemos criar outros para melhorar o desempenho das buscas.
- Criar um índice em um único campo
 - `db.usuarios.createIndex({ email: 1 })`
- Buscas pelo campo `email` serão muito mais rápidas:
 - `db.usuarios.find({ email: "joao@email.com" })`

ÍNDICES COMPOSTOS

Usando mais de um campo

ÍNDICES COMPOSTOS (VÁRIOS CAMPOS)

- Quando fazemos buscas com múltiplos critérios frequentemente, um índice composto pode ser mais eficiente.
- Criar um índice composto para **nome** e **idade**:
 - `db.usuarios.createIndex({ nome: 1, idade: -1 })`
 - Este índice ajuda buscas ordenadas pelo **nome** em ordem crescente e **idade** em ordem decrescente.
- Essa consulta usará o índice:
 - `db.usuarios.find({ nome: "Carlos" }).sort({ idade: -1 })`

ÍNDICES EM ARRAYS

ÍNDICES PARA BUSCA RÁPIDA EM ARRAYS

- Se um campo for um array e quisermos pesquisar dentro dele, podemos criar um índice multi-key.
- Criar um índice para um array:
 - **`db.pedidos.createIndex({ itens: 1 })`**
 - Isso melhora buscas em coleções onde **`itens`** é um array.

ÍNDICES EM CAMPOS TEXTUAIS

ÍNDICES EM CAMPOS TEXTUAIS

- Se precisamos buscar palavras em campos de texto, o índice textual é útil.
- Criar um índice para busca textual:
 - `db.produtos.createIndex({ descricao: "text" })`
- Agora podemos buscar palavras dentro desse campo:
 - `db.produtos.find({ $text: { $search: "notebook" } })`
 - Retorna todos os produtos cuja **descricao** contém "notebook".

ÍNDICES GEOESPACIAIS

ARMAZENAGEM DE DADOS DE GEOLOCALIZAÇÃO

- Se armazenarmos coordenadas geográficas, podemos criar índices geoespaciais (2dsphere).
- Criar um índice geoespacial:
 - `db.locais.createIndex({ localizacao: "2dsphere" })`
 - Agora podemos buscar locais próximos de um ponto específico.

QUANTO ESPAÇO UM ÍNDICE OCUPA?

QUANTO ESPAÇO UM ÍNDICE OCUPA?

- Se quisermos ver o espaço total ocupado pelos índices de uma coleção, usamos:
 - `db.usuarios.totalIndexSize()`
 - Saída: 5242880
 - Significa que os índices ocupam 5MB (5242880 bytes) na coleção **usuarios**.

DETALHANDO O ESPAÇO USADO POR ÍNDICES DE UMA COLEÇÃO

- **`db.usuarios.stats().indexSizes`**
- Saída esperada:

```
{  
  "_id_": 4096000,  
  "email_1": 1146880,  
  "nome_1_idade_1": 2293760  
}
```
- Explicação:
 - **`_id_`** → 4MB (MongoDB cria esse índice automaticamente).
 - **`email_1`** → 1.1MB (índice no campo **`email`**).
 - **`nome_1_idade_1`** → 2.2MB (índice composto em **`nome`** e **`idade`**).

COMO SABER SE O ÍNDICE ESTÁ SENDO USADO?

COMO SABER SE UM ÍNDICE ESTÁ SENDO USADO?

- Podemos analisar como uma consulta está sendo processada com **`.explain("executionStats")`**
- Exemplo sem índice:
 - **`db.usuarios.find({ email: "joao@email.com" }).explain("executionStats")`**
 - Se **`totalKeysExamined`** for 0 e **`totalDocsExamined`** for o total de documentos da coleção, significa que a consulta não está usando um índice.
- Exemplo com índice:
 - **`db.usuarios.find({ email: "joao@email.com" }).hint({ email: 1 }).explain("executionStats")`**
 - Se **`totalKeysExamined`** for baixo e **`totalDocsExamined`** também, significa que o índice foi utilizado com sucesso.

PARA QUE SERVE HINT() ?

- O MongoDB escolhe o melhor índice para executar uma consulta. No entanto, em alguns casos, ele pode fazer uma escolha subótima, e o `.hint()` permite forçar o uso de um índice que sabemos ser melhor.
- Caso de Uso:
 - Testar qual índice performa melhor
 - Evitar que o MongoDB use um índice “menos bom”

EXEMPLO DE USO DO HINT()

- Crie os índices:
 - `db.pedidos.createIndex({ email: 1 })`
 - `db.pedidos.createIndex({ cliente: 1, status: 1 })`
- Consulta:
 - `db.pedidos.find({ email: "joao@email.com" }).explain("executionStats")`
- Forçando o uso do índice `cliente_1_status_1`, podemos usar `.hint()`:
 - `db.pedidos.find({ email: "joao@email.com" }).hint({ cliente: 1, status: 1 }).explain("executionStats")`

REMOVENDO UM ÍNDICE

REMOVENDO ÍNDICES

- Se um índice não estiver sendo usado, podemos removê-lo para economizar espaço:
 - `db.usuarios.dropIndex("email_1")`
- Para listar os índices existentes de uma coleção:
 - `db.usuarios.getIndexes()`

AVALIANDO O DESEMPENHO DE ÍNDICES

AVALIANDO O DESEMPENHO DE CONSULTAS

- O comando `.explain("executionStats")` permite analisar como uma consulta está sendo executada, mostrando se um índice foi utilizado e qual foi o custo da operação.
- É essencial para otimizar consultas e entender a performance dos índices.

COMO USAR .EXPLAIN()?

- Se buscarmos por um campo **sem um índice**, o MongoDB **faz um full collection scan** (varredura completa):
 - `db.usuarios.find({ email: "joao@email.com" }).explain("executionStats")`

SAÍDA MÉTODO .EXPLAIN()

```
{  
  "executionStats": {  
    "totalKeysExamined": 0,  
    "totalDocsExamined": 100000,  
    "executionTimeMillis": 235  
  }  
}
```



- **Explicação:**
 - **totalKeysExamined: 0** → Nenhum índice foi usado.
 - **totalDocsExamined: 100000** → Todos os 100.000 documentos foram analisados.
 - **executionTimeMillis: 235** → Levou 235ms, um tempo relativamente alto.

TESTANDO O MESMO EXEMPLO, AGORA USANDO ÍNDICE

- Criação do Índice:
 - `db.usuarios.createIndex({ email: 1 })`
- Agora, executamos `.explain("executionStats")` novamente:
 - `db.usuarios.find({ email: "joao@email.com" }).explain("executionStats")`

MUDANÇA APÓS O USO DE ÍNDICES

```
{  
  "executionStats": {  
    "totalKeysExamined": 1,  
    "totalDocsExamined": 1,  
    "executionTimeMillis": 2  
  }  
}
```



- **Explicação:**

- **totalKeysExamined: 1** → O MongoDB encontrou o email pesquisado usando o índice.
- **totalDocsExamined: 1** → Apenas 1 documento foi lido, e não 100.000 como antes.
- **executionTimeMillis: 2** → A consulta agora leva apenas 2ms, uma melhora drástica!

CONCLUSÃO

- Índices aceleram buscas, mas consomem memória e afetam escrita.
 - ✓ Use índices simples para buscas frequentes em um campo.
 - ✓ Use índices compostos para consultas que filtram múltiplos campos.
 - ✓ Use índices textuais para buscas por palavras-chave.
 - ✓ Sempre monitore com `.explain()` para evitar índices desnecessários.

EXERCÍCIOS

EXERCÍCIO

- Crie uma coleção de 100 mil documentos e registre o tempo de consulta sem índices e depois com índices:

```
for (let i = 0; i < 100000; i++) {  
  db.usuarios.insertOne({  
    nome: `Usuario${i}`,  
    email: `usuario${i}@email.com`,  
    idade: Math.floor(Math.random() * 80) + 18  
  });  
}
```

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Plataforma **Google Classroom**
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@cps.sp.gov.br

